

强化学习接入推进路线图

文档目的

这份文档讨论前工程下一步该做什么、先做什么、做到什么算完成。

当前工程已经不是单纯的地图演示了，已经具备一部分适合接入强化学习的基础能力：

- 有时间推进与仿真步进
- 有巡逻/打击任务
- 有红蓝视角切换
- 有敌情航迹 `ContactTrack`
- 有通信/数据链相关字段
- 有燃油、雷达、武器、航迹等单位属性

但它还没有真正成为一个可训练的 RL 环境。最关键的原因不是“算法还没选”，而是还缺少完整的对局定义、初始部署规则、蓝方可观测接口、动作抽象、动作与观测边界、时间粒度、成本口径、随机性口径、状态表示方式、迷雾规则、RL 扮演角色选择、终局判定和训练包装层。

当前工程现状

从代码结构看，当前工程更像“连续仿真原型”，还不是“封闭任务环境”。

已经具备的基础：

- `ui/map_canvas.py` 已经支持单步推进和持续推进，仿真主循环清晰。
- `controllers/combat_controller.py` 已经有探测、交战、武器结算、任务驱动航路更新。
- `core/perspective.py` 已经区分 `god / blue / red` 视角。
- `core/contact.py` 已经有敌情航迹、置信度衰减、过期机制。
- `engine/communications.py` 已经有通信链路、数据链、链路退化的基础模型。
- `core/scenario.py` 已经把 `fuel / radar / comms / datalink / weapons` 等字段挂到单位上。

当前还没有形成闭环的部分：

- 没有统一的“裁判”模块来定义开始、结束、胜负、得分。
- 没有明确的双方初始部署规则，例如蓝方从哪里开始、红方从哪里开始、哪些位置允许随机化。
- 没有明确的“蓝方观测构建器”，现在的视角切换更多偏 UI。
- 没有明确的观测边界，例如哪些信息属于蓝方可见、哪些只属于裁判真值、哪些经过共享链路后才能可见。
- 没有明确 RL 在第一阶段应该扮演哪一层角色：任务级指挥官、编队级控制者，还是单单位驾驶员。
- 没有明确的动作边界，例如 agent 能下达任务命令、编队命令，还是直接控制单机机动。
- 没有明确的时间粒度，例如仿真每秒更新一次，但 agent 是每 1 秒、5 秒还是 10 秒做一次决策。
- 没有明确的成本口径，例如损失一架预警机和损失一架普通战机应如何区分价值。
- 没有明确“同样动作在不同敌情下得到不同结果”的随机性口径。
- 没有明确状态表示是偏离散、偏连续，还是结构化混合表示。
- 没有把迷雾和信息不确定性正式写成环境规则，现在更多还是视角展示能力。
- 没有 RL 训练环境包装层，例如 `reset()`、`step()`、`reward`、`done`。
- 没有任务阶段系统，例如出动、巡逻、搜索、接敌、打击、返航、任务完成。

- 燃油、通信、雷达等字段已经存在，但还没有完整地驱动任务失败、返航和资源约束。
- 缺少一套稳定的最小训练场景，当前更偏演示场景。

论文对本项目的直接启发

下面这张表只保留和当前项目推进直接相关的结论，不做泛泛综述。

论文	核心问题	对本项目的直接启发	当前是否立刻需要
1708.04782 StarCraft II Learning Environment	如何把复杂 RTS 包装成 RL 环境	先定义 <code>observation / action / reward / done</code> ，再谈算法	是
2403.17533 超视距空战 RL 环境	如何把空战问题做成可训练环境	先收缩任务类型和场景边界，不要一开始做全能沙盒	是
1507.06527 DRQN	部分可观测下如何利用历史信息	蓝方不能只看当前帧，需要历史敌情和记忆输入	是
1902.04043 SMAC	如何把大 RTS 收缩成局部多单位协同任务	先做“小任务环境”，再做大场景	是
rashid18a QMIX	多单位协同如何训练	训练可集中、执行必须分散，适合蓝方多单位协同	之后需要
1705.08926 COMA	多智能体奖励归因	以后若每个单位一个 agent，需要解决谁该拿奖励	之后需要
2103.01955 MAPPO/PPO	多智能体 cooperative baseline	想先跑通 baseline，PPO 系方法更现实	之后需要
2309.11247 分层空战 RL	高层任务控制与低层机动如何拆分	你的工程更适合先做“任务级指挥官”，不适合直接做微操	是

结论很明确：你现在最需要借鉴的，不是 AlphaStar 那种超大规模训练工程，而是“如何先把这个项目包装成一个小而清晰的部分可观测任务环境”。

你现在具体还缺少什么

下面这个表是当前最关键的缺口列表。它不是功能愿望单，而是 RL 接入前必须逐步补齐的结构。

缺口	为什么重要	建议产出物	优先级
对局定义	没有开始、结束、胜负，就没有 episode	<code>core/judge.py</code> 或 <code>core/outcome.py</code>	P0
初始部署规则	不定义蓝红双方从哪里开始，环境边界就不稳定	<code>deployment_zones</code> 与 <code>spawn rules</code>	P0
任务模板	没有固定任务类型，环境边界会一直漂	护航 / 拦截 / 区域巡逻 / 打击任务模板	P0

缺口	为什么重要	建议产出物	优先级
观测边界	不定义可见与不可见信息，就会泄漏真值	observation boundary	P0
蓝方观测构建器	不能让 agent 直接读真值	core/observation.py	P0
动作抽象	直接控制每秒航向和速度太难	task-level action space	P0
动作边界	不先定控制层级，动作空间会无限膨胀	action contract	P0
时间粒度	不明确决策频率，训练接口和 credit assignment 会失真	decision tick	P0
成本口径	不明确不同损失的价值，reward 会失真	score / cost model	P0
随机性口径	不明确环境是确定性还是随机性，训练预期会混乱	stochastic outcome rules	P0
状态表示方式	不明确状态是离散、连续还是混合，后续 baseline 难以统一	state representation note	P0
RL 角色层级定义	不先定 agent 扮演哪一层，观测和动作空间都会发散	agent role definition	P0
任务阶段系统	现在任务只有“挂上去”，没有阶段推进	mission_phase 与阶段转换规则	P1
资源约束闭环	燃油、弹药、通信、雷达没有完整影响行为	RTB / low fuel / lost comms 规则	P1
训练环境包装层	没有 Gym 风格接口就难以训练	rl_env.py 或 envs/ 目录	P1
场景族	只有演示场景，不利于训练和评估	data/scenarios/rl/	P1
评估指标	没有指标就无法比较策略优劣	胜率 / 存活率 / 任务完成率 / 平均损失	P1
baseline	没有 baseline 就无法验证环境是不是可学	scripted policy + PPO baseline	P2

最关键的一句话是：你现在缺的不是“更高级的算法”，而是“让这个项目先变成一个定义清楚的小型 POMDP 环境”。

可借鉴星际争霸的环境设计原则

星际争霸给这个项目最重要的启发，不是“大模型”或“大算力”，而是环境口径的定义方式。

建议借鉴下面这些原则：

- 迷雾不是 UI 效果，而是环境规则的一部分。
- 观测永远小于真值，策略输入不能直接等于裁判输入。
- 动作要分层，先做高层任务命令，不直接暴露所有微操。
- 仿真时间和决策时间可以分开，底层可以按秒推进，高层 agent 不一定每秒决策。

- 同样的动作在不同侦察质量、敌情隐藏程度和交战概率下，可以得到不同结果，这不是 bug，而是环境特征。
- 奖励不只看输赢，还要反映任务成本、平台价值和关键资源损失。
- 状态表示不必一开始就强行全离散化，可以先采用“结构化混合表示”。

强化学习扮演角色的方向建议

在这个项目里，RL 可以扮演三个层级的角色，但它们的难度和前置条件差别很大。

角色层级	控制对象	需要的动作粒度	优点	风险	当前建议
任务级指挥官	整体任务与编队任务分配	低	最容易先收束环境；最贴近现有任务系统	细节机动表达能力较弱	第一阶段推荐
编队级控制者	单个编队或机群	中	比任务级更灵活，能体现协同机动	动作和状态复杂度明显上升	第二阶段再做
单单位驾驶员	单机或单舰	高	控制最细，理论上上限最高	训练难度极高，环境会很快失控	当前不推荐

当前最合理的方向是：第一版把 RL 明确定位成“任务级指挥官”。

原因不是它最强，而是它最适合当前工程的成熟度：

- 你已经有任务系统雏形，适合在此基础上继续抽象。
- 你现在最缺的是对局定义和任务闭环，而不是飞行微操。
- 任务级动作更容易和部分可观测敌情、通信、燃油约束结合。
- 先把任务级环境跑通，后面再向编队级下钻，比一开始就做单机控制更稳。

因此，这份路线图默认：

- 第一阶段：RL 扮演任务级指挥官
- 第二阶段：如有需要，再扩展到编队级控制者
- 第三阶段：只有在前两层稳定后，才考虑单单位驾驶员

不建议现在就做的事

为了避免项目发散，下面这些方向现在都不应该优先：

- 不要一开始就做全地图、全军种、全任务类型的通用对抗环境。
- 不要一开始就做每个单位每秒微操级别的动作空间。
- 不要一开始就做红蓝双方都用 RL 自博弈。
- 不要先追求非常复杂的奖励塑形。
- 不要先把时间花在更多零散 UI 上。

先做出一个小而闭环、能训练、能评估的环境，后面再扩展。

推荐推进顺序

推荐按四个阶段推进，每个阶段都要有明确产出，不要并行铺太多线。

阶段 1：把仿真原型收束成任务环境

目标：定义一场“局”。

建议先固定一个最小任务：

- 蓝方任务：护送 1 个高价值单位穿越指定区域
- 红方任务：在时间限制内发现并拦截该单位
- 时长：例如 15 到 20 分钟仿真时间
- 终局：护送成功、护送目标被毁、时间耗尽、蓝方失去任务能力

这一阶段的重点不是训练，而是先建立统一裁判规则。

本阶段应完成：

- 定义 `scenario seed` 和部署规则
- 定义蓝方部署区、红方部署区、允许的随机出生点范围
- 定义高价值目标、护航单元、拦截单元的初始位置生成方式
- 定义迷雾规则和敌情初始已知范围
- 定义任务开始条件
- 定义任务结束条件
- 定义胜负判定
- 定义基础得分项
- 定义成本口径，例如高价值目标、护航机、预警机、拦截机、弹药消耗各自如何计价

阶段 2：把“蓝方视角”真正从 UI 规则变成训练输入

目标：建立真正的部分可观测接口。

当前已有 `god / blue / red` 视角和 `ContactTrack`，这是很好的基础，但它们还没有被整理成策略可消费的 `observation`。

本阶段应完成：

- 区分 `world_state` 和 `blue_observation`
- 定义明确的观测边界：哪些信息蓝方天然可见，哪些必须侦察后可见，哪些永远只给裁判
- 蓝方 `observation` 只包含：
 - 己方单位状态
 - 已探测敌情
 - 敌情时间戳和置信度
 - 共享链路状态
 - 任务状态
- 明确哪些信息只能给裁判，不能给 `agent`
- 把迷雾、敌情滞后、共享延迟都写成 `observation` 规则的一部分

阶段 3：建立任务级动作空间，而不是微操空间

目标：把动作难度降到能训练。

推荐第一版不要让 `agent` 直接控制速度、航向角、细粒度转向，而是先给它任务级动作：

- 指派哪支编队前往哪个任务区
- 继续巡逻 / 返航 / 拦截 / 护航 / 打击
- 雷达开机 / 关机
- 是否允许共享链路
- 是否优先保护高价值目标

如果第一版动作太细，训练很可能直接卡住。

阶段 4：建立最小训练闭环

目标：让环境可以真正跑训练。

本阶段应完成：

- 明确 RL 第一版扮演任务级指挥官
- 定义动作边界：允许下达哪些命令，不允许下达哪些细粒度操作
- `reset()`
- `step(action)`
- `reward`
- `done`
- `info`
- scripted baseline
- PPO baseline
- 一个最小评估脚本

做到这一步，项目才算真正进入“RL 工程阶段”。

最小可行 RL 环境定义（建议版本）

这一节是当前最建议落地的 MVP，不追求完美，只追求先跑通。

任务名称

蓝方护航，红方脚本拦截。

环境边界

- 单场景
- 单任务类型
- 有时间上限
- 蓝方是 RL agent
- 红方先用规则脚本

观测边界

第一版建议像星际争霸一样，把“可观测信息”和“真值信息”严格拆开。

建议方向：

- 裁判真值可以知道全图所有单位真实位置、状态和任务。
- 蓝方 agent 只能看到己方状态、已探测敌情、共享来的敌情、以及敌情置信度。

- 未探测到的红方单位，对蓝方 observation 来说应该等价于不存在。
- 已探测但失去接触的红方单位，应该只保留最后已知位置、发现时间和置信度衰减。

也就是说，迷雾和信息不确定性要成为环境定义的一部分，而不是显示层附加效果。

动作边界

第一版建议严格限制 RL 可控制的层级，避免动作空间过大。

建议方向：

- 允许：任务分配、编队前出、编队返航、任务姿态切换、探测策略切换。
- 不允许：单机逐秒转向、单机油门曲线、逐发导弹微操、逐点路径绘制。

这和星际争霸中常见的“高层策略决策先于低层执行细节”思路一致。

时间粒度

建议把“仿真推进粒度”和“策略决策粒度”分开定义。

建议方向：

- 仿真底层可以仍然按 1 秒推进，保持现有引擎简单清晰。
- RL 决策不一定每秒做一次，可以考虑每 5 秒或 10 秒做一次高层决策。
- 底层多个 1 秒步骤共同对应一次 agent action 的执行窗口。

这样做的好处是：

- 降低动作频率
- 降低 credit assignment 难度
- 更符合任务级指挥官的决策节奏

成本口径

第一版不要只用“赢了 +1，输了 -1”。

建议方向：

- 高价值目标损失应远重于普通护航平台损失。
- 预警机、电子战平台、加油平台等高价值支援单位应有更高成本权重。
- 弹药和燃油消耗可以先作为次级成本，不要一开始权重过高。
- 任务是否完成应始终高于单纯击毁数量。

也就是说，reward 的口径要体现“任务价值优先”，而不是简单的“杀敌数优先”。

同样动作的不同结果

第一版就应该接受：同样的动作，在不同敌情、不同迷雾、不同命中概率下，结果可能不一样。

建议方向：

- 这类差异应被视为环境正常属性，而不是系统不稳定。
- 隐藏信息会导致“同样前出侦察”在不同局里发现不同敌情。

- 交战概率会导致“同样发起拦截”在不同局里得到不同损失结果。

这和星际争霸非常像：相同宏观动作在不同侦察信息和敌方部署下，本来就会产生不同结果。

状态表示方式

第一版不建议强行把所有状态全做成纯离散。

建议方向：

- 内部世界状态可以继续保持连续或近连续表示，例如位置、速度、时间、油量、置信度。
- 暴露给 agent 的 observation 可以采用结构化混合表示：
 - 一部分是连续数值特征
 - 一部分是离散任务状态
 - 一部分是敌情存在与否、雷达开关、链路状态这类布尔或类别特征

这比一开始强行完全离散化更贴近当前工程，也更容易逐步试验 baseline。

双方初始部署

第一版不要把单位初始位置写成完全自由摆放，而要写成“部署规则”。

建议方向：

- 蓝方单位从蓝方部署区开始，例如机场、港口、母舰周边区域或预设巡逻箱体。
- 红方单位从红方部署区开始，例如对向机场、前沿巡逻区或拦截待命区。
- 高价值目标不建议每局固定在单一点，而建议在一个小范围内部署区内按随机种子生成。
- 护航编队和拦截编队也不建议完全固定死点，而建议在各自部署区内做可控随机化。

这样做有三个好处：

- 每局可复现，因为有 `scenario seed`
- 每局不完全一样，训练不会死记一个脚本
- 部署规则比手工摆点更容易扩展成场景族

第一版推荐用下面这种定义方式：

- `deployment_zone_blue`
- `deployment_zone_red`
- `spawn_rule_hva`
- `spawn_rule_blue_escort`
- `spawn_rule_red_interceptor`

强化学习扮演的角色

第一版建议明确写死：

- RL 扮演蓝方任务级指挥官

这意味着 agent 负责的是：

- 让哪支编队去哪个区域
- 何时前出侦察

- 何时回收或返航
- 何时切换护航、拦截、搜索等任务姿态
- 何时开启或关闭更激进的探测策略

这也意味着 agent 暂时不负责：

- 单机每秒钟的转向细节
- 单机速度曲线控制
- 逐发武器微操

把这一点写死很重要，因为它会直接决定 **observation** 和 **action** 的边界。

蓝方 observation

建议由以下几部分组成：

1. 己方单位状态
包括位置、速度、航向、剩余燃油、雷达状态、武器余量、当前任务。
2. 已知敌情
包括最后发现位置、发现时间、敌情置信度、是否来自共享链路。
3. 编队与网络状态
包括链路是否可达、关键节点是否在线、共享是否退化。
4. 任务状态
包括护航目标当前位置、距目标区距离、剩余时间、当前阶段。

蓝方 action

第一版建议离散化：

- 护航目标继续前进
- 护航编队向 A 区机动
- 护航编队向 B 区机动
- 前出侦察
- 回收侦察
- 执行拦截
- 全队返航
- 雷达静默
- 雷达开机

reward

第一版不要设计太复杂，先保证奖励方向正确：

- 护航成功：大正奖励
- 护航目标被毁：大负奖励
- 发现敌方高威胁目标：小正奖励
- 蓝方关键护航单位被毁：中到大负奖励
- 无效机动导致脱离任务区太久：小负奖励

- 时间耗尽但目标未达成：负奖励

done

出现以下任一条件即结束：

- 护航目标进入目标区
- 护航目标被摧毁
- 仿真时间耗尽
- 蓝方失去继续执行任务的能力

接下来 1 到 2 周该怎么推进

这一节是最实际的执行建议，优先追求把一条线做通。

时间段	目标	具体任务	产出
第 1-2 天	定义一场“局”	明确任务模板、初始部署、蓝红起点、迷雾规则、胜负、终局、评分项	一份任务定义 md + 裁判结构草案
第 3-4 天	定义蓝方观测	列出哪些信息可见、哪些不可见、哪些可共享、哪些只属于裁判真值	observation 设计文档
第 5-6 天	定义动作空间	先明确 RL 扮演任务级指挥官，再定义动作边界、时间粒度，不碰微操	action space 设计文档
第 7-9 天	建环境包装层	设计 <code>reset / step / reward / done</code>	RL env 骨架
第 10-12 天	做最小场景	固定一个蓝护航红拦截场景	<code>data/scenarios/rl/</code> 示例
第 13-14 天	做 baseline	先 scripted，再 PPO	baseline 脚本与初步结果

如果时间更紧，可以把第一周目标压缩成一句话：
先把“蓝方护航、红方脚本拦截”的最小任务环境定义清楚，不写训练代码也没关系。

资料目录建议怎么用

你现在根目录下已经有：

- `论文/论文`
- `论文/笔记`
- `论文/代码`

这个结构可以继续保持，不需要重构。

建议使用方式：

- `论文/论文`：只放原文 PDF，不做修改。
- `论文/笔记`：每篇论文提炼 3 个问题：
 - 这篇论文解决了什么问题？

- 对我的工程最有用的是什么？
- 哪一条结论会直接变成代码结构？
- 论文/代码：只放复现代码、参考实现、实验脚本，不放项目主代码。

不要把论文笔记直接写成大段总结，最好总是回到“会改变我项目哪一个结构”。

推荐新增的工程产出物

为了让下一步更顺，建议后续围绕下面这些文件或模块推进：

- docs/rl/ 或继续放在 docs/superpowers/specs/：保存 RL 相关设计文档
- core/judge.py：胜负、终局、评分裁判
- core/observation.py：蓝方/红方观测构建器
- core/action_space.py：任务级动作定义
- core/mission_phase.py：任务阶段与阶段转换
- envs/rl_env.py：训练环境包装层
- data/scenarios/rl/：固定训练与评估场景
- tools/eval_policy.py：策略评估脚本

当前的最优先事项

如果只允许做一件事，那么最应该先做的是：

先定义一个最小任务环境，而不是先写训练代码。

更具体一点，就是先把下面四件事定死：

1. 蓝方任务是什么
2. 红方任务是什么
3. 蓝方从哪里开始，红方从哪里开始
4. RL 在第一阶段扮演谁
5. 蓝方到底能看见什么，什么看不见
6. 蓝方到底能控制什么，什么不能控制
7. agent 多久决策一次
8. 同样动作为什么可能得到不同结果
9. 什么情况下结束
10. 什么情况下算蓝方赢

这几件事一旦清楚，后面的 observation、action、reward、done 都会自然长出来；如果这些事不清楚，后面无论读多少论文、试多少算法，工程都会继续发散。